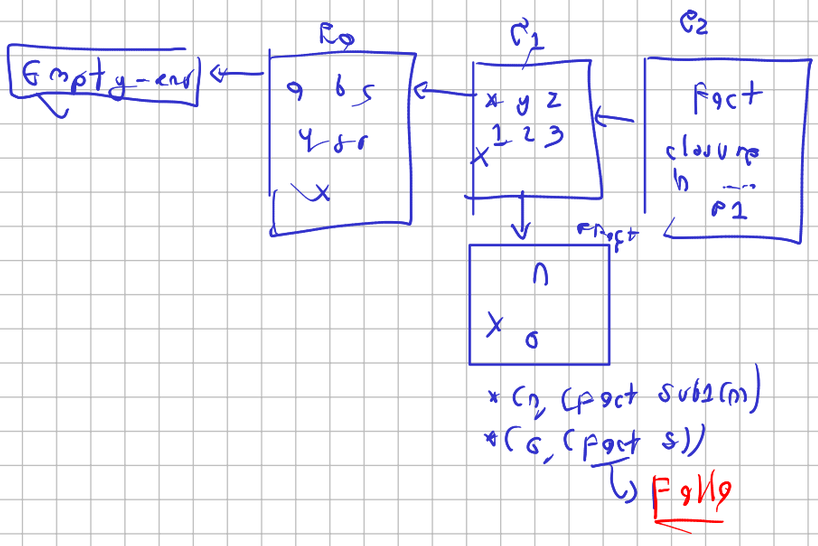
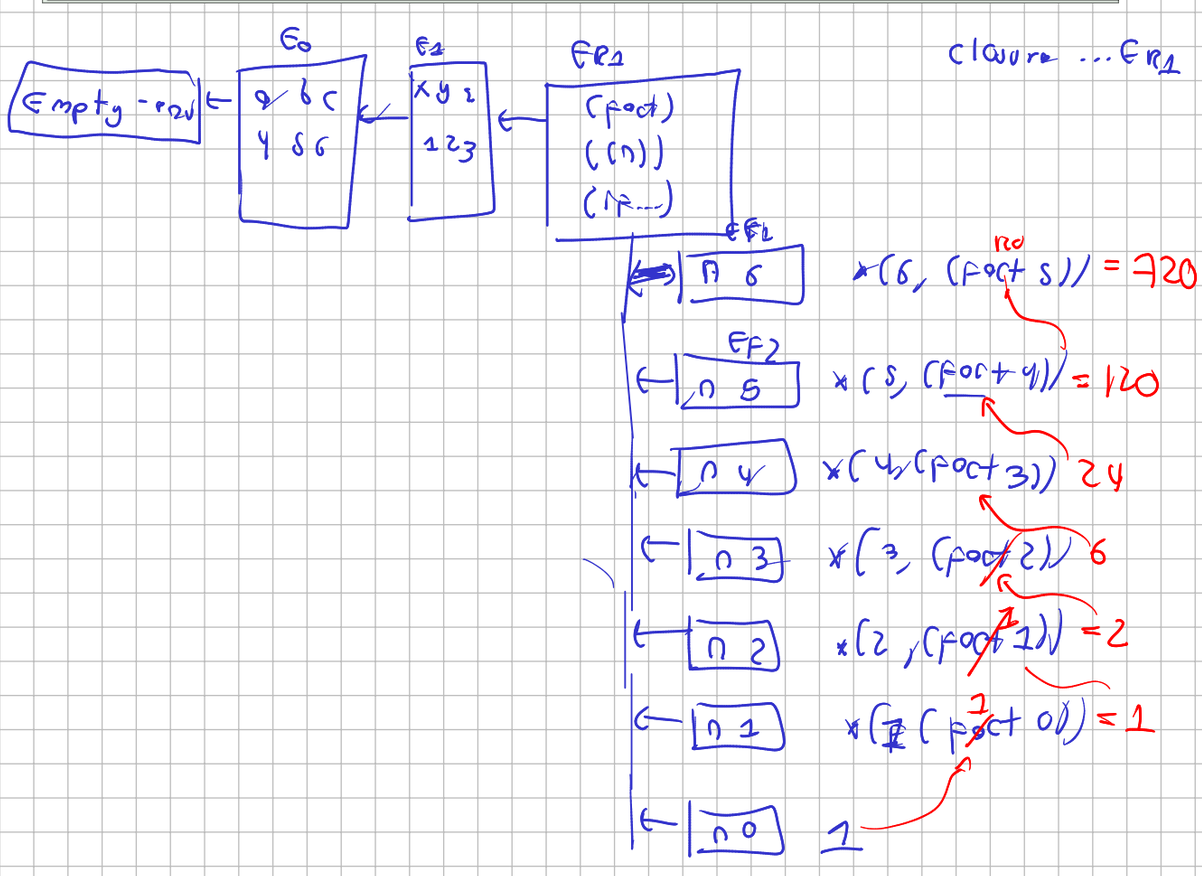
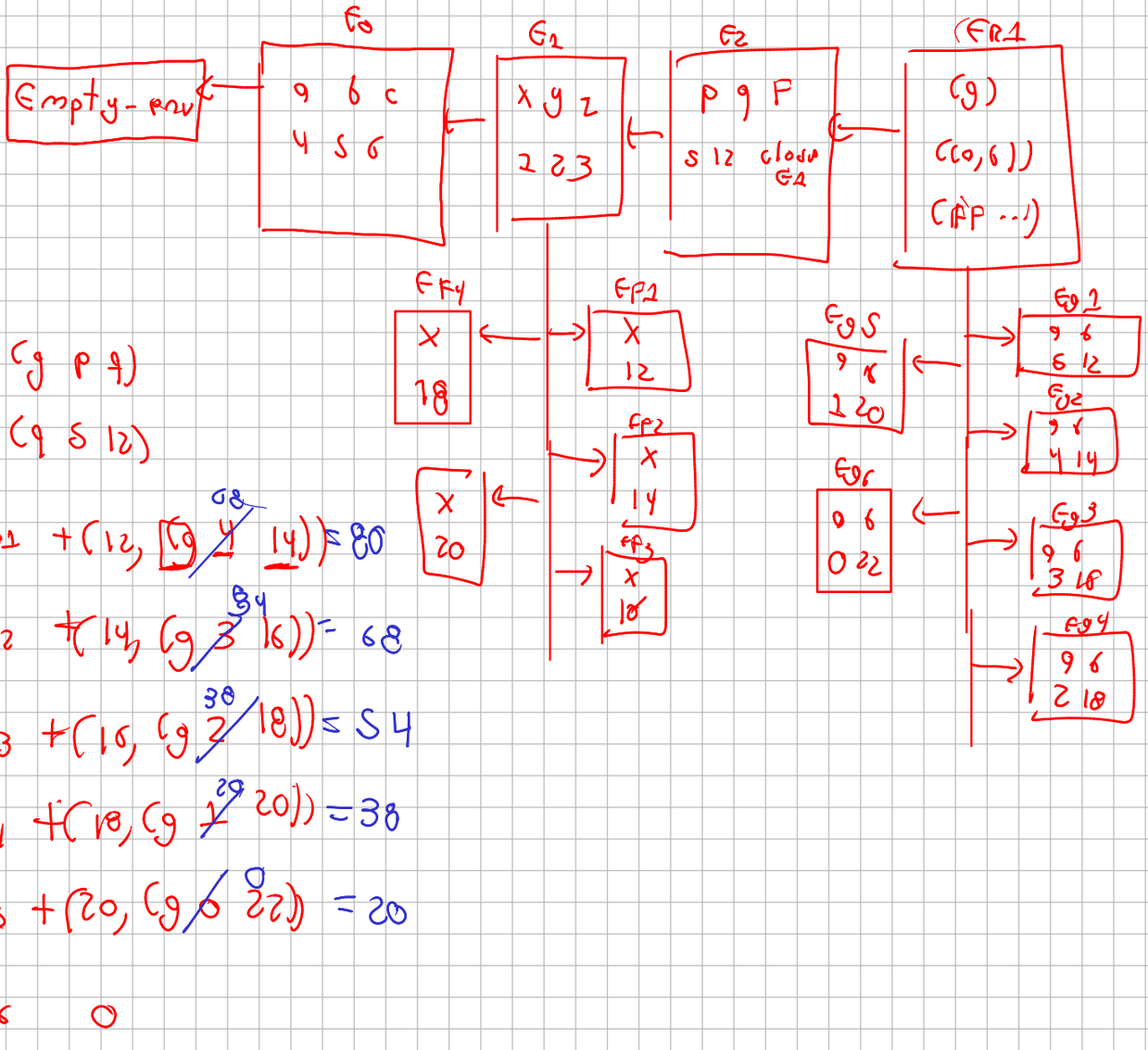
```
let
  fact = proc(n) if ==(n,0) then 1 else *(n, (fact sub1(n)))
in
  (fact 6)
```




```
letrec
  fact = proc(n) if ==(n,0) then 1 else *(n, (fact sub1(n)))
in
  (fact 6)
```



```
let
  p = 5
  q = 12
  f = proc(x) +(x,2)
in
  letrec
    g(a,b) = if >(a,0) then +(b, (g sub1(a) (f b))) else 0
  in
    (g p q)
```



(g p q)

(g 5 12)

Eg1 +(12, (g 4 14)) = 80

Eg2 +(14, (g 3 16)) = 68

Eg3 +(16, (g 2 18)) = 54

Eg4 +(18, (g 1 20)) = 38

Eg5 +(20, (g 0 22)) = 20

Eg6 0

Recordar, un ambiente se genera en 3 condiciones

1) en let .. in (despues del in)
2) en letrec ... in (despues del in)
3) Cuando invoco un procedimiento

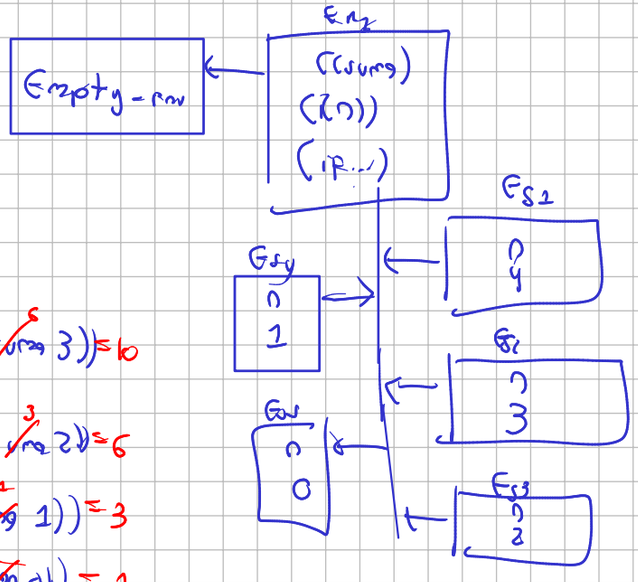
letrec cuenta(n) = if ==(n,0) then 0 else add1((cuenta sub1(n))) in (cuenta 3)

add1(cuenta 2))
add1(add1(cuenta 1)))
add1(add1(add1 (cuenta 0))
add1(add1(add1 0))))
3

letrec cuenta(n) = if ==(n,0) then 0 else add1((cuenta sub1(n))) in (cuenta 3)

(cuenta 3) (cuenta 2) (cuenta 1) (cuenta 0) ---> 4
sub1(n) --> 3 sub1(3) sub1(2) sub1(1) -->3
if ==(n 0) --> 4 ===(3 0) ==(2 0) ==(1 0) ==(0 0) --> 4
Total 11

letrec suma(n) = if ==(n,0) then 0 else +(n, (suma sub1(n))) in (suma 4).



← Vacío

} 10

$E_{R1}(suma, y)$

E_{s1} +(y, (suma 3)) = 10

E_{s2} +(3, (suma 2)) = 6

E_{s3} +(2, (suma 1)) = 3

E_{s4} +(1, (suma 0)) = 1

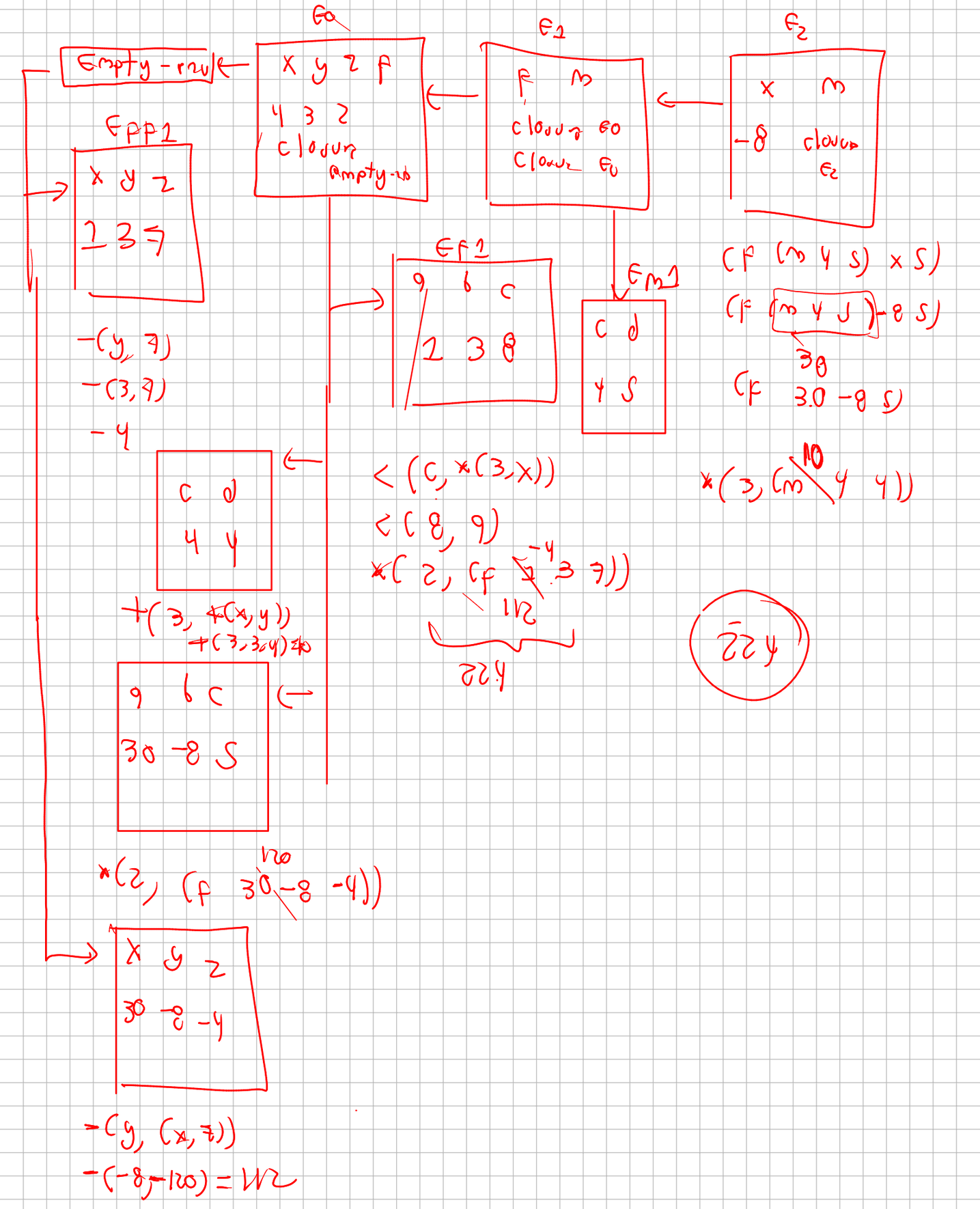
E_{s5} 0 ✓

```
let
   f = proc(a, b, c)
     if c<*(3,x) then *(2, (f a b +(x,b)))
     else 4
   m = proc(c, d) +(3, +(x,y))
in
   let
     x = (f 1 3 8)
     m = proc(c, d) if d>0
                 then *(3, (m c -(d,1)))
                 else c
     in
       (f (m 4 5) x 5)
```

Considere la siguientes expresiones en el lenguaje visto en el curso (procedimientos), con ambiente inicial:

env0 con identificadores **(x y z f)** y valores **(4 3 2(closure'(x y z) -(y,*(x,z)) empty-env))**

```
letrec
   f = proc(a, b, c) =
     if c<*(3,x) then *(2, (f a b +(x,b)))
     else 4
   m = proc(c, d) =+(3, +(x,y))
in  x=(f 1 3 8)
   letrec
     x = (f 1 3 8)
     m = proc(c, d) =if d>0
                 then *(3, (m c -(d,1)))
                 else c
     in
       (f (m 4 5) x 5)
```



```
letrec
  f (a, b, c) =
      if >(c,*(3, x))
      then *(2, (f a b + (x, b)))
      else 4
in
let
  m = proc (c, d)
       +(3, +(x, y))

  x = (f 1 3 8)

in
  letrec

    m (c, d) =
       if >(d,0)
       then *(3, (m c -(d, 1)))
       else c

  in
   (f (m 4 5) x 5)
```

Considere la siguientes expresiones en el lenguaje visto en el curso (procedimientos), con ambiente inicial:

env0 con identificadores **(x y z f)** y valores **(4 3 2(closure'(x y z) -(y,*(x,z)) empty-env))**

